



UNIVERSITY OF SALERNO

Computer Science Department

Master's Degree in Computer Science

GRADUATION THESIS

Comparing handwritten and AI-generated microbenchmarks for performance regression detection

SUPERVISOR

Prof. Dario Di Nucci

Dr. Antonio Trovato

University of Salerno

CANDIDATE

Oussama Boukhachem

Matricola: 0522501392

Anno Accademico 2024-2025

This Master Thesis was realized in the



Dedica o citazione

Abstract

Performance benchmarking is the process of testing how well a software system works by running it under controlled conditions to measure its speed and resource usage. Therefore, this practice is essential for detecting performance regressions, which are sub-optimal design choices that cause the system to use more resources or take longer to respond without actually crashing.

When performance benchmarking is conducted over smaller software components, such as single Java methods or specific algorithms, it is referred to as performance microbenchmarking. Although frameworks such as Java Microbenchmark Harness (JMH) help developers automate this process to collect detailed metrics, still, developing microbenchmarks manually is a very difficult and time-consuming task that requires deep domain expertise. Therefore, creating reliable performance tests by hand remains a significant technical burden for most software teams. A critical question remains: can Ai-generated microbenchmarks detect performance regressions as effectively as handwritten benchmarks?

This thesis presents a practical evaluation and comparison of handwritten and AI-generated microbenchmarks for regression detection. We will be focusing on Apache Ignite, which is a distributed database system, examining three distinct commits from its development history. For each of commit, we analyze 16 benchmark pairs each consisting of a handwritten benchmark and its Ai twin.

To reach our objective, we have executed each handwritten microbenchmark and its AI twin on both the original version and a modified version of the system. Specifically, the modification consisted of an artificially generated delay in the software to simulate performance regressions.

Our results show that both the handwritten and the AI-generated benchmarks were highly effective, achieving a perfect 100% detection rate for all tested performance regressions. But while both types of tests found every performance regression, the AI-generated versions were more sensible to smaller changes than the original software versions. This research confirms that modern AI tools can produce high-quality and reliable code for automating performance testing in complex software environments.

Contents

List of Figures	iii
List of Tables	iv
1 Introduction	1
1.1 Research Context	1
1.2 Motivations and Goals	2
1.3 Achieved Results	3
1.4 Thesis Structure	4
2 Background and Related Works	6
2.1 Research Background	6
2.1.1 Software Performance Testing	6
2.1.2 Microbenchmarking	7
2.1.3 Java Microbenchmarking Harness (JMH)	8
2.1.4 Performance Regression Detection	10
2.2 Large Language Models (LLMs)	11
2.2.1 Prompt Engineering	12
2.3 Related Works	13
3 Methodology	15

3.1	Goal and Research Questions	15
3.2	Research Design and Case Study	16
3.2.1	Choice of the Subject System	17
3.2.2	Selection of Software Versions	17
3.3	Benchmark Selection and AI Generation	18
3.3.1	Handwritten Benchmark Selection	18
3.3.2	AI Generation and Prompt Engineering	23
3.4	Performance Regression Injection	26
3.4.1	Injection Strategy	27
3.4.2	Scope of Injection	27
3.5	Evaluation Metrics	28
3.5.1	Detection Definition	28
3.5.2	Statistical Analysis: Hierarchical Bootstrap	28
3.5.3	Performance Change: Percentage Slowdown	29
3.6	Execution Environment	30
3.6.1	Hardware and Software Configuration	30
3.6.2	System-Level Optimizations	30
3.6.3	JMH Configuration	31
4	Results	33
4.1	RQ1 — Detection Effectiveness	33
4.2	RQ2 — Measurement Sensitivity	34
4.3	RQ3 — Generalization and Consistency	38
5	Threats to Validity	41
5.1	Construct Validity	41
5.2	Internal Validity	42
5.3	External Validity	42
5.4	Conclusion Validity	42
6	Conclusions	43
	Bibliography	45

List of Figures

4.1	Detection rates for handwritten and AI-generated benchmarks across all three Apache Ignite versions.	34
4.2	Performance impact (percentage slowdown) measured by handwritten (blue) and AI-generated (purple) benchmarks, with 95% confidence intervals, <code>commit1_b038730(v1.8.0)</code>	36
4.3	Performance impact (percentage slowdown) measured by handwritten (blue) and AI-generated (purple) benchmarks, with 95% confidence intervals, <code>commit2_feba953(v2.0.0)</code>	37
4.4	Performance impact (percentage slowdown) measured by handwritten (blue) and AI-generated (purple) benchmarks, with 95% confidence intervals, <code>commit3_160dab0(v2.2.0)</code>	38
4.5	Performance Impact Distribution by Benchmark Category	39
4.6	$Average_{RegressionImpact_{byCategory}}$	40

List of Tables

2.1	Performance Measurement Modes	10
3.1	Apache Ignite versions selected as experimental subjects	18
4.1	Detection rates per commit	34
4.2	Measured performance impact: handwritten vs AI-generated (% slow-down, point estimate)	35

CHAPTER 1

Introduction

1.1 Research Context

In the modern software engineering landscape, delivering high-quality applications is no longer just about ensuring that the code works without errors.

Performance has become a critical non-functional requirement that directly affects how users perceive a product and how much it costs for a company to operate its servers [1, 2]. For the aforementioned reasons, developers should treat software efficiency as a fundamental priority to remain competitive in nowadays environments where speed and reliability are essential for success. As software systems constantly evolve through new features and updates, there is a recurring risk of introducing "performance regressions." These are sub-optimal design choices that do not cause the system to crash but instead make the software slower or more resource-intensive [3, 4].

Because these issues are often hidden and do not impact functional correctness, traditional unit tests usually fail to identify them. Therefore, detecting these regressions early in the release pipeline is necessary to prevent performance degradation from reaching the end users.

In this context, performance benchmarking is the process of testing how well a software system works by running it under controlled conditions to measure its speed and resource consumption. When benchmarking is focused of testing smaller components, such as single Java methods or specific algorithms, it is referred to as microbenchmarking. Java Microbenchmark Harness (JMH) is the de-facto standard framework for developing Java microenchmarks. It consists of an annotation-based framework, which allows developers to easily build, run, and analyze Java microbenchmarks [5].

1.2 Motivations and Goals

Implementing accurate performance microbenchmarks is a major challenge because any Java code is subject to internal optimizations by the Java Virtual Machine (JVM)[6, 7]. So, since a microbenchmark simply runs a Java method, its initial performance would be extremely unstable and unreliable due to Just-In-Time (JIT) compilation[5, 3]. Therefore, developers must define "warm-up iterations" to perform these initial, "useless" executions until the JVM finishes optimizing the code to a steady state [6, 7]. That's why the accuracy of a performance test depends heavily on discarding these early fluctuations to reach a stable state of peak performance.

However, the current state-of-practice is far from perfect because developers usually rely on their own intuition and expertise to configure these warm-up periods [6, 5]. Recent research has demonstrated that even human experts frequently fail to accurately estimate the end of the warm-up phase, which leads to a significant waste of testing time or the collection of poor-quality results [3, 6]. Therefore, there is a clear need for more automated and rigorous approaches to ensure that performance measurements are both valid and efficient. Beyond timing issues, writing microbenchmarks requires defending the code against aggressive compiler optimizations like "dead code elimination"[8]. If a benchmark is not written with expert knowledge, the JVM might detect that a result is not used and simply skip the execution of the code being tested[8]. So, creat-

ing high-quality performance suites is a time-consuming and expensive task that many software teams struggle to maintain. This difficulty is even higher in large-scale distributed systems like Apache Ignite, where concurrency and complex data operations make performance regressions very hard to identify. These challenges provide the primary motivation for this thesis. With the recent rise of Large Language Models (LLMs) that can generate code from natural language descriptions, there is a promising opportunity to automate the creation of these complex microbenchmarks [9, 3]. However, it is not yet known if AI-generated code can handle the strict technical requirements of frameworks like the Java Microbenchmark Harness (JMH). Therefore, this research aims to conduct a practical evaluation to see if AI can match the effectiveness of human developers in finding performance regressions.

The main goal of this study is to compare handwritten microbenchmarks with their AI-generated "twins" using Apache Ignite as a case study [3]. We focus on detecting artificial regressions across three major versions of the software under test (SUT) (v1.8.0, v2.0.0, and v2.2.0).

1.3 Achieved Results

To ensure the highest scientific rigor, we conducted our measurements in a strictly controlled execution environment designed to minimize background interference. So, this setup allowed us to isolate the software's behavior and eliminate random fluctuations that could distort the performance data [6, 5]. Therefore, the Java Microbenchmark Harness (JMH) was configured following established instructions from previous academic studies to ensure that all tests were executed under stable and reproducible conditions[7]. That's why, by using this standardized approach, we could accurately measure the sensitivity of both handwritten and AI-generated tests in identifying efficiency issues within a real-world system like Apache Ignite [3].

The most significant achievement of this research is the confirmation that AI-generated microbenchmarks are remarkably effective at finding performance

bugs. Our experimental results show that both the handwritten code and their AI-generated "twins" achieved a 100% detection rate across all three tested versions of the software, v1.8.0, v2.0.0, and v2.2.0 [3]. But, while both types of tests successfully identified every performance regression, the AI-generated benchmarks proved to be the superior choice since the AI versions measured a larger difference in performance between the original software versions and the ones containing regressions. Our data reveals that the median performance impact recorded by the AI was consistently higher than that of the handwritten code across all tested versions. AI "twins" provide a clearer signal of efficiency issues, making them even more sensitive and reliable than benchmarks written by human experts.

1.4 Thesis Structure

The rest of this thesis is organized into several chapters that describe the research process, the technical implementation, and the final findings. So, the document follows a logical path from the theoretical background to the practical conclusions.

Chapter 2 - Background and Related Works: This chapter provides the fundamental technical concepts required to understand the study. It defines performance testing, microbenchmarking, and the Java Microbenchmark Harness (JMH). It also introduces the architecture of Apache Ignite and the current state of Large Language Models (LLMs) for code generation. Finally, it explores existing research in automated testing to highlight the current research gaps.

Chapter 3 - Methodology: In this chapter, we detail the research design and the experimental workflow. We explain how the handwritten benchmarks were selected and how their "AI twins" were generated through specific prompt engineering. Therefore, this section establishes the scientific framework for the comparative study, including the regression injection method and the statistical approach using hierarchical bootstrap analysis.

Chapter 4 - Implementation: This chapter focuses on the practical execution of

the experiment. It describes the build system configuration for Apache Ignite and the custom automation scripts developed for Ubuntu. So, we provide a detailed look at how the benchmarking environment was optimized to ensure stable and reproducible measurements.

Chapter 5 - Results and Analysis: Here, we present the quantitative findings of our experiments. This includes the analysis of the 100% detection rate and the magnitude of the performance changes found by both benchmark types. Therefore, this chapter uses statistical evidence and visualizations to compare the effectiveness of human and AI code.

Chapter 6 - Threats to Validity: This chapter examines the factors that could limit the reliability or the general utility of our findings. We identify threats and describe the mitigation strategies we applied during the study.

Chapter 7 - Conclusions and Future Work: The final chapter summarizes the main contributions of the thesis and provides final remarks. Therefore, we offer practical recommendations for the industry and suggest new directions for future research in AI-driven performance engineering.

CHAPTER 2

Background and Related Works

This chapter provides the fundamental technical concepts and the theoretical framework required to understand the research conducted in this thesis. It defines the principles of software performance testing, the specific methodology of microbenchmarking, and the industry-standard tools used for Java applications. This chapter also introduces and discusses the current state of the art of Large Language Models and the techniques used to guide them in generating specialized technical code.

2.1 Research Background

2.1.1 Software Performance Testing

Performance testing is a fundamental branch of software quality engineering that aims to systematically evaluate a system's responsiveness, stability, and resource usage under specific workloads [1, 5]. While functional testing verifies if the code produces the correct output, performance testing focuses on how well the software performs its tasks. So, this discipline is necessary to ensure that applications can meet the high expectations of modern users and maintain operational efficiency. Therefore, developers treat performance as a critical

non-functional requirement that directly impacts both user satisfaction and the costs of running hardware [1].

In a professional development lifecycle, performance testing is categorized into several types depending on the technical goals of the evaluation. For instance, load testing measures the behavior of a system under expected traffic, whereas stress testing pushes the application to its absolute limits to identify where it might fail [1]. Other methods, such as soak testing or spike testing, are used to find memory leaks or stability issues during sudden bursts of activity [1]. That's why software teams must use a variety of these methodologies to understand the full performance profile of a complex system. So, a comprehensive testing strategy is the only way to guarantee that a product remains reliable as it evolves over time.

Traditionally, these performance evaluations were conducted on a large scale against entire systems in production-like environments. However, these "macro-benchmarks" are often very time-consuming to set up and difficult to integrate into fast-paced development cycles [8, 5]. Therefore, the industry has shifted toward more granular methods that can isolate small code fragments to achieve higher precision. That's why we must examine how testing operates at a more detailed level to provide the fine-grained metrics needed for optimization. Furthermore, this transition requires specialized automation frameworks that can manage the technical complexities of modern execution environments.

2.1.2 Microbenchmarking

Microbenchmarking is a form of lightweight performance testing that targets the smallest units of software, such as individual Java methods or specific algorithms [6, 5, 8]. Unlike macro-benchmarks that evaluate an entire system under heavy user load, these tests evaluate code fragments in isolation [8, 1]. This practice is "performance unit testing", it applies a highly granular approach to non-functional requirements [8, 5]. It allows developers to obtain precise metrics like throughput and execution time without the background noise of the rest of the application [8, 7].

The main objective of this technique is to identify performance regressions early in the development cycle before the code is released to production [4, 3]. In complex distributed systems, even a tiny change in a frequently used method can significantly degrade the overall responsiveness and increase operational costs [1, 5]. Microbenchmarking is considered a critical tool for maintaining high quality in performance-critical software [8]. By using these focused tests, teams can verify that new updates do not introduce sub-optimal design choices efficiency bugs that traditional functional tests would typically miss [3, 4]. It provides a necessary safety net for continuous performance assessment.

However, implementing reliable microbenchmarks in Java is technically difficult due to the dynamic nature of the Java Virtual Machine (JVM). At the beginning of a test run, the code is executed slowly, but the JVM eventually identifies "hot" paths and applies Just-In-Time (JIT) compilation to optimize them into efficient machine code. This process creates a warmup , where performance measurements are highly unstable and potentially misleading [6, 7, 5]. Results should only be collected once the software has reached a "steady-state" of peak efficiency. Developers must carefully design their benchmarks to handle these non-deterministic behaviors to ensure that the data is statistically valid. Therefore, writing effective microbenchmarks is a specialized discipline that requires a deep understanding of both the measurement methodology and the underlying platform architecture [6, 7].

2.1.3 Java Microbenchmarking Harness (JMH)

In the Java development community, the Java Microbenchmark Harness (JMH) is recognized as the standard framework for building and analyzing performance tests at a method level [5]. JMH was developed under the OpenJDK project to provide a reliable infrastructure that handles internal technical challenges, such as reaching a performance steady state through warmup iterations [6, 5]. It is a common misunderstanding to believe the framework itself can fully isolate code from the surrounding execution environment [3, 5].

Even with an ideal JMH configuration, measurements can still be influenced

by background system processes, hardware fluctuations, or operating system events [6, 7]. So, while JMH is essential for managing Java Virtual Machine instability, achieving high-precision results still requires developers to manually ensure a strictly controlled environment to prevent data corruption from external noise [3, 1].

The framework operates through a sophisticated annotation-based system. Therefore, practitioners can easily configure their benchmarking environment directly within the source code by using specific annotations [8]. One of the most useful features of the harness is its support for different Benchmark Modes, which allow the user to decide exactly what metric should be measured.

So whether a developer needs to know the total throughput which measures the number of operations per unit of time or the average execution time that measures the average time it takes for a single operation to execute, JMH provides a specific mode to satisfy that requirement.

JMH handles the non-deterministic behavior of the JVM by distinguishing between an operation, which is a single execution of the code under test, and an iteration, which is the period of time where the framework runs as many operations as possible. So, in a single 100-millisecond iteration, JMH might execute a method millions of times while recording the time for each individual operation to calculate the final throughput or average time metrics. Therefore, the framework uses Warmup to reach a stable steady-state performance before recording any real data [6, 7]. That's why using separate JVM processes called "Forks" is standard practice, as it ensures each test run is isolated and not affected by the memory layout of previous runs.

The following table provides a summary of these primary modes :

Table 2.1: Performance Measurement Modes

Mode	Description
Throughput	Measures the number of operations per unit of time (e.g., operations per second).
Average Time	Measures the average time it takes for a single operation to execute.
Sample Time	Measures the time for each operation and provides statistical percentiles and distribution data.
Single Shot Time	Measures the time for a single execution, typically used for testing "cold" performance without warmup.
All	Executes all the modes above simultaneously to provide a comprehensive performance profile.

Writing effective benchmarks also requires specific defenses against aggressive optimizations like "dead code elimination," where the JVM deletes code it mistakenly thinks is unused. So, JMH provides specialized objects called Blackholes to "consume" results and keep the code active in the eyes of the compiler [8]. Another pitfall is "constant folding," where the JVM replaces a predictable calculation with a fixed result because it knows the constants will always produce the same outcome. Therefore, developers are encouraged to use `@State` classes to provide dynamic inputs initialized outside the benchmark method. In JMH, a `@State` class is a container object used to hold benchmark data (for example arrays, maps, counters, or parameters) with a defined lifecycle and scope (such as per thread or shared across the benchmark). This is why setup logic is separated from the timed method, so measurements remain more stable and representative. [5].

2.1.4 Performance Regression Detection

A performance regression is a bad design choice that makes the system to act slower or consume more resources even if it still produces the correct output. Unlike traditional functional bugs that lead to obvious errors or crashes,

these regressions are silent and only degrade the quality of the user experience [4, 3]. Therefore, it is necessary to identify various technical causes for these degradations, such as inefficient algorithmic changes, resource leaks, or locking contention in concurrent code [6]. Understanding these patterns is key to optimizing a performance testing campaign and ensuring the software remains competitive.

However, detecting regressions is notoriously difficult because performance measurements are often "noisy" and affected by factors beyond the developer's control [6, 5]. Background operating system activities, non-deterministic hardware fluctuations, and virtual machine warm-up issues can make results inconsistent from one test run to another [6, 7]. Practitioners must rely on rigorous statistical methods, to distinguish between random measurement variance and a real degradation in code efficiency [3, 5].

The ultimate goal of regression detection is to find these efficiency bugs early in the development cycle before they reach production and impact the final users. That's why establishing a solid performance baseline for comparison is a vital step for any reliable benchmarking project [1].

By combining precise measurement tools with strong statistical analysis, developers can build a safety net that protects the system from performance decay as the codebase evolves.

2.2 Large Language Models (LLMs)

Large Language Models (LLMs) are advanced artificial intelligence systems primarily based on the Transformer architecture, which uses self-attention mechanisms to process and generate complex sequences of data [9]. These models have evolved rapidly over the last few years, moving from basic text completion to sophisticated code generation across various programming languages [9, 10]. So, they are no longer just experimental tools but are becoming integrated into the professional software development lifecycle to assist with day-to-day engineering tasks. Therefore, understanding their internal capabili-

ties is essential for determining how they can be applied to more specialized technical domains.

Modern models, such as GPT-4 and Claude, are trained on massive datasets that include millions of open-source repositories from platforms like GitHub [9, 5]. This extensive training allows the models to recognize not only standard syntax but also structural patterns, common algorithms, and even specific framework best practices. That’s why they have shown a remarkable ability to automate repetitive coding tasks and generate functionally correct code from natural language descriptions [11, 9]. So, developers can often improve their productivity by using these models to generate boilerplate code or initial drafts of unit tests. Using LLMs for specialized performance engineering tasks, such as writing microbenchmarks, introduces a higher level of complexity. Unlike functional code, performance tests must follow strict rules to avoid measurement errors like dead code elimination or lack of JVM warmup. Therefore, while the AI can generate the syntax of a benchmark, it must be guided carefully to ensure the resulting code is scientifically valid for regression detection [5, 3]. That’s why it is necessary to examine the specific techniques used to interact with these models to produce high-quality technical output. So, the following section will explore the role of instruction design in optimizing AI-generated code.

2.2.1 Prompt Engineering

Prompt engineering is the systematic practice of designing and refining inputs to guide Large Language Models toward generating high-quality and contextually relevant responses [12]. So, it is no longer just about asking a simple question, but about crafting a structured instruction that the model can follow with technical precision [9]. Therefore, understanding how to structure these inputs is essential for obtaining specialized results, such as source code that must follow the strict rules of a specific framework. That’s why prompt engineering is considered a primary skill for developers looking to integrate AI into their professional workflow.

In technical domains, two prompt patterns are especially effective: *Persona*

prompting and *Few-shot prompting*. In *Persona prompting*, the model is explicitly assigned a professional role (for example, “expert Java/JMH performance engineer”), so its reasoning and output are constrained toward domain-specific conventions, terminology, and priorities. In *Few-shot prompting*, the prompt includes one or more concrete input–output examples, so the model can infer the expected structure, abstraction level, and coding style before generating new content. Therefore, combining these two patterns improves both relevance and consistency, which is why they are widely adopted in software-oriented generation workflows [12, 3].

So, by acting as an expert developer and seeing examples of correct code, the model is much more likely to produce functionally accurate results. That’s why practitioners also use “system contexts” to define the behavior and format of the AI’s response before the actual task is even presented [11]. Therefore, a well-designed prompt acts as a filter that reduces the risk of hallucinations and ensures the generated code is technically sound.

In the context of this research, prompt engineering was used to bridge the gap between human-written benchmarks and AI-generated versions. That’s why a structured approach was adopted to generate the “AI twins” used in our comparative study, providing the model with the original handwritten source code and specific instructions to replicate its logic using JMH annotations [5]. This involved guiding the model to maintain semantic equivalence while following microbenchmarking best practices, such as proper resource management and state handling. So, the prompt served as the critical link between existing developer knowledge and the automated generation of performance tests. Therefore, the quality of our experimental data depends directly on the effectiveness of these prompting strategies in replicating human expertise.

2.3 Related Works

The intersection of Large Language Models and software engineering has become a major focus of recent academic research. This section reviews existing

studies regarding the quality of AI-generated code, the application of AI in software testing, and current advancements in automating performance evaluations. So, by examining these fields, we can identify the specific research gaps that this thesis aims to address.

AI-Generated Code Quality Studies Current literature has extensively evaluated how well Large Language Models can generate functionally correct source code. Research indicates that while models like GPT-4 can outperform traditional static analyzers in identifying and fixing logic errors, they often struggle with specialized technical requirements [11, 10]. While AI is excellent at general programming tasks, its ability to follow the strict architectural rules required for performance testing remains a subject of investigation. Some studies suggest that LLMs may inadvertently introduce security vulnerabilities or ignore framework-specific best practices if they are not prompted correctly [10, 9]. That’s why evaluating the structural integrity of AI-generated microbenchmarks is a critical step in determining their reliability for industry use.

CHAPTER 3

Methodology

This chapter describes the research design and the experimental workflow adopted in this study. It presents the goals and research questions that guide the work, explains the choice of the case study, and details the process used to select and generate the benchmarks. Furthermore, it describes the regression injection strategy, the evaluation metrics, and the execution environment that were set up to ensure that results are both valid and reproducible.

3.1 Goal and Research Questions

The primary goal of this study is to conduct an empirical comparison between handwritten JMH microbenchmarks and their AI-generated equivalents, evaluating their effectiveness in detecting software performance regressions [3, 4]. So, rather than simply asking whether AI can generate syntactically correct Java code [9], the central question is whether the resulting benchmarks are actually useful for engineering purposes. Therefore, this research focuses on three specific and measurable dimensions of effectiveness that together form a complete picture of the problem.

The first dimension concerns detection capability: the ability of a benchmark to

determine whether a performance regression exists in the system under test [13]. The second dimension concerns measurement sensitivity: the strength and clarity of the signal that a benchmark produces when a regression is present [6]. The third dimension concerns consistency: whether the results are stable across different software versions and operation types. That’s why the following three research questions were formulated to drive the experimental design.

Q RQ₁. *Can AI-generated microbenchmarks detect performance regressions with the same success rate as handwritten microbenchmarks?*

This question evaluates whether AI-generated code can match the fundamental capability of human-written tests in identifying the presence of a performance bug [8]. Therefore, the answer will determine if AI-generated benchmarks are functionally viable as a replacement or complement to manually written ones.

Q RQ₂. *While both benchmark types may detect a regression, is there a significant difference in the magnitude of performance degradation measured by AI-generated versus handwritten code?*

Beyond mere detection, this question investigates how clearly each benchmark type quantifies the impact of a regression [14]. So, even if both types detect the same bug, one may produce a much larger and clearer performance signal, making the issue easier to identify with confidence.

Q RQ₃. *Does the effectiveness of AI-generated benchmarks remain consistent across different software versions (v1.8.0, v2.0.0, v2.2.0) and various operation types (Cache, Future, Increment)?*

This question examines whether the results depend on a specific version of the software or a particular type of operation [1]. Therefore, by testing across multiple commits and different benchmark categories, we can determine whether the conclusions of this study generalize beyond a single narrow context [15].

3.2 Research Design and Case Study

This study follows an empirical research design based on controlled experimentation. The core approach consists of running the same set of benchmarks

against two versions of the same software, one of which contains an artificially injected performance regression, and then comparing the results obtained by handwritten and AI-generated benchmarks.

3.2.1 Choice of the Subject System

Apache Ignite was selected as SUT for several important reasons. It is a large-scale, open-source, distributed in-memory computing platform that is actively maintained and widely used in performance-critical production environments [2]. Systems of this nature are exactly the kind of software where performance regressions have real consequences, since even small efficiency losses in core operations can cascade into significant degradation at scale. Testing on a system that genuinely depends on high performance gives the research strong practical relevance. Furthermore, it includes a collection of official JMH microbenchmarks as part of its source code repository written by the developers of the system itself, that's a genuine human expertise applied to performance testing. Using them as the handwritten baseline for this study ensures that the comparison is fair and grounded in professional-quality code [13]. The open-source nature of Apache Ignite means that its full development history is publicly accessible. It was possible to select specific commits from its Git history and build the exact state of the software at each point in time independently, without any ambiguity about which version of the code is being tested [4].

3.2.2 Selection of Software Versions

To ensure that the findings of this study are not tied to a single specific state of the software, three distinct major versions of Apache Ignite were selected correspond to three commits from the official repository, chosen to cover a meaningful span of the project's development timeline.

Table 3.1: Apache Ignite versions selected as experimental subjects

Commit ID	Version	Date
b038730	v1.8.0	2017-11-08
feba953	v2.0.0	2016-11-14
160dab0	v2.2.0	2017-08-01

The first version, corresponding to commit `b038730`, represents Apache Ignite v1.8.0. The second, commit `feba953`, corresponds to v2.0.0, which introduced significant changes to core internal components. The third, commit `160dab0`, corresponds to v2.2.0.

The number of available and working benchmarks differs slightly across versions. Commit 1 (v1.8.0) provided 2 benchmark pairs, while commits 2 and 3 (v2.0.0 and v2.2.0) each provided 7 benchmark pairs, for a total of 16 handwritten benchmarks and 16 AI-generated twins. The reasons for this difference are purely technical: some benchmarks present in later versions did not exist in the earlier commit, and others were excluded because they exhibited unstable or hanging execution behavior during smoke testing [6].

3.3 Benchmark Selection and AI Generation

This section describes how the handwritten benchmarks were chosen to form the baseline of the experiment, and how their AI-generated counterparts were produced. Therefore, understanding both processes is essential to evaluate the fairness and scientific validity of the comparison.

3.3.1 Handwritten Benchmark Selection

The handwritten benchmarks used in this study were not written specifically for this research. Instead, they are the official JMH microbenchmarks that are already included in the Apache Ignite source code repository and maintained by its developers. Therefore, they represent real-world, professional-quality

performance tests rather than benchmarks created under controlled academic conditions. This is an important distinction, as it means the baseline is as authentic as possible [8].

The selection process consisted of identifying all JMH benchmarks present in each of the three chosen commits and then performing a smoke test for each one. A smoke test is a quick preliminary run with minimal configuration, designed to verify that a benchmark compiles, starts, and produces output without hanging or crashing. Therefore, only benchmarks that passed this initial test were included in the final experiment. Any benchmark that caused the JVM to hang or failed to terminate cleanly was excluded from the study to avoid polluting the results with unreliable measurements [7].

After applying this selection process, the surviving benchmarks were grouped into four functional categories based on the type of Apache Ignite operation they exercise:

Cache Operations: benchmarks that measure the performance of basic distributed cache read and write operations (`get` and `put`), which are the most fundamental operations in any in-memory data grid.

Future and Asynchronous Operations: benchmarks that evaluate the latency and throughput of asynchronous task completion and listener registration, covering methods such as `testSimpleGet`, `testSimpleGetWithListener` and `completeFutureGet`.

Increment Operations: benchmarks that compare the performance of atomic counters against thread-local increment strategies, covering methods `increment` and `threadLocalIncrement`.

Park and Notify Operations: benchmarks that measure thread synchronization primitives, including `park` and `condition` methods.

Therefore, this categorization ensures that the benchmark suite covers a broad range of operation types rather than being biased toward a single area of the system.

The following listing shows one of the handwritten benchmarks used as a baseline, specifically `JmhCacheBenchmark`, which measures the performance of cache `get` and `put` operations:

```
package org.apache.ignite.internal.benchmarks.jmh.cache;
import org.apache.ignite.IgniteDataStreamer;
import org.apache.ignite.cache.CacheAtomicityMode;
import org.apache.ignite.cache.CacheWriteSynchronizationMode;
import org.apache.ignite.internal.benchmarks.jmh.runner.
JmhIdeBenchmarkRunner;
import org.apache.ignite.internal.benchmarks.model.IntValue;
import org.openjdk.jmh.annotations.Benchmark;
import org.openjdk.jmh.profile.GCProfiler;
import java.util.concurrent.ThreadLocalRandom;

/**
 * Put benchmark.
 */
@SuppressWarnings("unchecked")
public class JmhCacheBenchmark extends JmhCacheAbstractBenchmark {
    /**
     * Set up routine.
     *
     * @throws Exception If failed.
     */
    public void setup() throws Exception {
        super.setup();
        IgniteDataStreamer<Integer, IntValue> dataLdr = node.dataStreamer(cache.getName());
    }
}
```

Listing 1: JmhCacheBenchmark.java — handwritten cache benchmark Part 1 of 3

```
for (int i = 0; i < CNT; i++)
dataLdr.addData(i, new IntValue(i));
dataLdr.close();
System.out.println("Cache populated.");
}

/**
 * Test PUT operation.
 * @throws Exception If failed.
 */
@Benchmark
public void put() throws Exception {
    org.openjdk.jmh.infra.Blackhole.consumeCPU(1000000L);
    int key = ThreadLocalRandom.current().nextInt(CNT);
    cache.put(key, new IntValue(key));
}

/**
 * Test PUT operation.
 * @throws Exception If failed.
 */
@Benchmark
public Object get() throws Exception {
    org.openjdk.jmh.infra.Blackhole.consumeCPU(1000000L);
    int key = ThreadLocalRandom.current().nextInt(CNT);
    return cache.get(key);
}

/**
 * Run benchmarks.
 * @param args Arguments.
 * @throws Exception If failed.
 */
public static void main(String[] args) throws Exception {
    run("put", CacheAtomicityMode.ATOMIC);
    run("get", CacheAtomicityMode.ATOMIC);
    run("put", CacheAtomicityMode.TRANSACTIONAL);
    run("get", CacheAtomicityMode.TRANSACTIONAL);
}

/**
 * Run benchmarks for atomic cache.
 * @param benchmark Benchmark name.
 * @param atomicityMode Atomicity mode.
 * @throws Exception If failed.
 */
private static void run(String benchmark, CacheAtomicityMode atomicityMode) throws
↳ Exception {
    run(benchmark, 4, true, atomicityMode, CacheWriteSynchronizationMode.PRIMARY_SYNC);
    run(benchmark, 4, true, atomicityMode, CacheWriteSynchronizationMode.FULL_SYNC);
    run(benchmark, 4, false, atomicityMode, CacheWriteSynchronizationMode.PRIMARY_SYNC);
}
```

Listing 2: JmhCacheBenchmark.java — handwritten cache benchmark Part 2 of 3

```
run(benchmark, 4, false, atomicityMode, CacheWriteSynchronizationMode.FULL_SYNC);
}

/**
 * Run benchmark.
 * @param benchmark Benchmark to run.
 * @param threads Amount of threads.
 * @param client Client mode flag.
 * @param atomicityMode Atomicity mode.
 * @param writeSyncMode Write synchronization mode.
 * @throws Exception If failed.
 */
private static void run(String benchmark, int threads, boolean client, CacheAtomicityMode
↳ atomicityMode,
CacheWriteSynchronizationMode writeSyncMode) throws Exception {
    String simpleClsName = JmhCacheBenchmark.class.getSimpleName();
    String output = simpleClsName + "-" + benchmark +
    "-" + threads + "-threads" +
    "-" + (client ? "client" : "data") +
    "-" + atomicityMode +
    "-" + writeSyncMode;
    JmhIdeBenchmarkRunner.create()
        .forks(1)
        .threads(threads)
        .warmupIterations(10)
        .measurementIterations(60)
        .benchmarks(simpleClsName + "." + benchmark)
        .output(output + ".jmh.log")
        .profilers(GCProfiler.class)
        .jvmArguments(
            "-Xms4g",
            "-Xmx4g",
            "-XX:+UnlockCommercialFeatures",
            "-XX:+FlightRecorder",
            "-XX:StartFlightRecording=delay=30s,
            dumpOnExit=true,settings=alloc,filename=" + output + ".jfr",
            JmhIdeBenchmarkRunner.createProperty(PROP_ATOMICITY_MODE, atomicityMode),
            JmhIdeBenchmarkRunner.createProperty(PROP_WRITE_SYNC_MODE, writeSyncMode),
            JmhIdeBenchmarkRunner.createProperty(PROP_DATA_NODES, 2),
            JmhIdeBenchmarkRunner.createProperty(PROP_CLIENT_MODE, client))
        .run();
}
```

Listing 3: JmhCacheBenchmark.java — handwritten cache benchmark Part 3 of 3

3.3.2 AI Generation and Prompt Engineering

Once the handwritten benchmarks were selected, the next step was to generate their AI equivalents. Therefore, the goal was not simply to ask an AI model to write a benchmark from scratch, but to produce a version that tests the exact same functionality as the original, while being written independently without copying any code.

To achieve this, GPT-4 was used as the AI model. GPT-4 was selected for several reasons. First, it was the most capable publicly available model for code generation at the time of this study, consistently outperforming alternatives on structured Java tasks [9]. Second, it is well suited to following complex multi-constraint prompts, which is important given that the prompt used here combined persona prompting, few-shot examples, and several explicit requirements simultaneously. Third, the main reference study on AI-generated benchmarks also used GPT-4 [3], which makes the methodological alignment between this work and the existing literature more direct. Therefore, GPT-4 was the natural choice for this experiment.

The interaction was structured using a carefully designed prompt. The quality of the output depends heavily on how the prompt is constructed, since LLMs are highly sensitive to the framing, context, and level of detail provided in the input [12]. A multi-part prompt structure was adopted, combining two well-established techniques in prompt engineering.

The first technique is **Persona prompting**, which consists of assigning the AI model a specific professional identity before giving it a task. GPT-4 was instructed to act as an expert Java developer specializing in JMH performance microbenchmarking. This framing encourages the model to apply domain-specific knowledge and conventions rather than producing generic Java code [12].

The second technique is **Few-shot prompting**, which involves providing the model with the original handwritten benchmark as a concrete example of the desired output format and quality. So, rather than describing what a JMH benchmark should look like in abstract terms, the prompt includes the full source code of the original benchmark along with explicit instructions about what

aspects must be preserved and what must be independently reimplemented [9]. The key requirements embedded in the prompt were the following: the AI must analyze the original benchmark and understand what operation it measures; it must rewrite the benchmark from scratch using different variable names, method structures, and code organization; it must maintain the same JMH annotations and measurement parameters; and it must follow best practices to avoid common pitfalls such as dead code elimination and improper state management [5, 3].

The following listing shows the complete prompt template used to generate the AI twins in this study:


```

# AI Microbenchmark Generation Prompt
## ROLE
You are an expert Java developer specializing in performance microbenchmarking using the
↳ JMH (Java Microbenchmark Harness) framework.
## TASK
Generate an AI-written equivalent of the provided handwritten JMH microbenchmark. The
↳ AI-generated benchmark must test the exact same functionality as the original, but
↳ should be written from scratch based on understanding the purpose and behavior of the
↳ original code.
## REQUIREMENTS
### 1. Functional Equivalence
- Analyze the original benchmark to understand what it measures (e.g., cache get/put
↳ operations, thread synchronization, etc.)
- Preserve the core testing logic: Same operations being measured, same data structures,
↳ same concurrency patterns
- Maintain JMH annotations: Use appropriate @Benchmark, @State, @Setup,
↳ @BenchmarkMode, etc.
- Keep the same scope and scale: If original uses ThreadLocalRandom for key selection,
↳ use similar approach
### 2. Code Independence
- Do NOT copy-paste the original code
- Write fresh implementations that achieve the same testing goals
- Use different variable names, method structures, and code organization where possible
↳ while maintaining clarity
- Keep the same package structure to ensure integration with the existing project
### 3. JMH Best Practices
- Use proper JMH annotations (@Benchmark, @State, @Setup, @TearDown, etc.)
- Include appropriate scope for state objects (e.g., Scope.Thread, Scope.Benchmark)
- Avoid dead code elimination (return results from benchmark methods when appropriate)
- Use proper warmup and measurement iterations if specified
### 4. Code Quality
- Clean, readable code with appropriate comments
- Professional naming conventions
- Proper exception handling
- Keep Apache License header (copy exactly from original)
### 5. Integration Requirements
- Must compile with the existing project structure
- Use the same imports and dependencies as available in the original
- Maintain compatibility with the benchmark runner infrastructure
## OUTPUT FORMAT

Provide:
1. The complete Java source code for the AI-generated benchmark class
2. File path where it should be saved (format:
↳ `src/main/java/org/apache/ignite/internal/benchmarks/jmh/ai/[OriginalName]AI.java`)
3. No explanations, just code
## ORIGINAL BENCHMARK CODE

```

Listing 4: Prompt template used for AI benchmark generation part 1 of 2

HERE WE CHANGED THE MICRO BENCHMARKING JAVA CODE EVERYTIME TO GENERATE THE AI TWIN.

GENERATION INSTRUCTIONS

1. Read and understand the original benchmark's purpose
2. Identify the core operation being measured
3. Generate a fresh implementation that tests the same operation
4. Ensure all JMH annotations are correctly applied
5. Verify the code would compile in the existing project context
6. Name the class by appending "AI" to the original class name (e.g., `JmhCacheBenchmark` → JmhCacheBenchmarkAI``
7. Place in package: `org.apache.ignite.internal.benchmarks.jmh.ai``

Listing 5: Prompt template used for AI benchmark generation part 2 of 2

All AI-generated benchmarks were placed in a dedicated package, `org.apache.ignite.internal.benchmarks.jmh.ai`, to keep them clearly separated from the original handwritten code. Therefore, each AI benchmark was named by appending the suffix `AI` to the original class name, for example `JmhCacheBenchmarkAI`. That's why it was always unambiguous which benchmark was handwritten and which was AI-generated throughout the entire experimental pipeline. Finally, every AI-generated benchmark was also smoke-tested before being accepted into the final experiment, applying the same quality filter used for the handwritten baselines.

3.4 Performance Regression Injection

To evaluate whether a benchmark can detect a performance regression, it is first necessary to create one. Therefore, this study uses a controlled injection approach: instead of searching for a real regression in the codebase, an artificial performance degradation is introduced manually and in a deterministic way. This strategy is widely used in performance testing research because it allows for precise control over the nature, location, and magnitude of the regression, which makes it possible to measure how sensitive each benchmark is to the exact same change [4, 13].

3.4.1 Injection Strategy

The regression was injected by inserting a single line of code at the beginning of each benchmarked method in the SUT. That line is a call to `Blackhole.consumeCPU(1000000L)`, which is a method provided by the JMH framework itself. Therefore, this call instructs the JVM to perform a fixed number of meaningless CPU operations, simulating a CPU-intensive delay without producing any useful result. That’s why the JVM cannot optimize it away through dead code elimination, making the injected slowdown both reliable and consistent across all runs [5].

The choice of `Blackhole.consumeCPU` is a clean injection that does not alter the logic or the data flow of the method being tested, only its execution time. So, the system continues to produce correct functional results, which mirrors exactly the nature of a real performance regression: a change that makes the code slower without breaking it [3].

The following listing shows the injection applied:

```
public void put() throws Exception {
    org.openjdk.jmh.infra.Blackhole.consumeCPU(1000000L);
    int key = ThreadLocalRandom.current().nextInt(CNT);
    cache.put(key, new IntValue(key));
}
public Object get() throws Exception {
    org.openjdk.jmh.infra.Blackhole.consumeCPU(1000000L);
    int key = ThreadLocalRandom.current().nextInt(CNT);
    return cache.get(key);
}
```

Listing 6: Example of regression injection inside a benchmark method

3.4.2 Scope of Injection

The injection was applied uniformly to all benchmark methods across all three Apache Ignite versions under test. Every handwritten benchmark and every AI-generated benchmark was executed against the exact same modified version

of the system, ensuring a completely level playing field for the comparison. Any difference in detection outcome or measured impact between the two benchmark types can be attributed solely to the design of the benchmark itself, and not to any variation in the regression they were exposed to.

The injection was performed on the source code of Apache Ignite itself, not inside the benchmark code. So, the benchmarks remained untouched, and the artificial slowdown was placed inside the internal methods that the benchmarks call during execution. Therefore, neither the handwritten nor the AI-generated benchmarks had any knowledge of or special adaptation to the injected regression. This design ensures that the detection is a genuine capability of the benchmark, not an artifact of how the code was written [8].

3.5 Evaluation Metrics

This section describes the measurement configuration used to collect performance data and the statistical methods applied to determine whether a benchmark successfully detected a regression.

3.5.1 Detection Definition

A regression is considered **detected** if the statistical analysis reveals a significant performance difference between the measurements collected on the clean version of the system and those collected on the injected version. Therefore, a simple comparison of average execution times is not sufficient: the difference must be statistically meaningful, large enough to exclude the possibility that it is caused by random noise in the measurements [15].

3.5.2 Statistical Analysis: Hierarchical Bootstrap

To assess whether an observed performance difference is statistically significant, this study uses the **hierarchical bootstrap** method, following the same approach

adopted in the reference study on AI-generated benchmarks [3]. The hierarchical bootstrap is a resampling-based technique that is specifically designed to handle the nested structure of JMH measurement data. In a standard JMH experiment, individual timing measurements are nested within forks, and forks are nested within the overall benchmark run. So, treating all measurements as independent, as a standard bootstrap would do, leads to underestimating the true variance of the experiment, because it ignores the correlation between measurements that come from the same JVM process. Therefore, the hierarchical bootstrap resamples at both levels simultaneously, first resampling forks and then resampling iterations within each selected fork, which produces confidence intervals that correctly reflect the variability of the full experiment.

The analysis was performed with **10,000 bootstrap iterations** and a **95% confidence interval**. A regression is declared detected when the confidence interval of the estimated performance difference between the clean and the injected version does not include zero. This criterion provides a clear and principled detection rule that is both statistically sound and easy to interpret [14].

3.5.3 Performance Change: Percentage Slowdown

Beyond detecting whether a regression exists, this study also measures the **magnitude** of the performance impact. The metric used for this purpose is the **percentage slowdown**, defined as:

$$\text{Percentage Slowdown} = \left(\frac{\hat{\mu}_{\text{injected}}}{\hat{\mu}_{\text{clean}}} - 1 \right) \times 100\% \quad (3.5.1)$$

where $\hat{\mu}_{\text{injected}}$ and $\hat{\mu}_{\text{clean}}$ are the bootstrap point estimates of the median execution time for the injected and clean versions, respectively. Therefore, a value of 5,000% means the injected version ran approximately fifty times slower. This metric was chosen because it is scale-independent and directly comparable across benchmark pairs regardless of their absolute execution times, making it well suited to compare handwritten and AI-generated benchmarks side by side.

side [3].

3.6 Execution Environment

Obtaining reliable benchmark results requires not only a correct experimental design but also a controlled and stable execution environment. Therefore, all experiments were run on a dedicated server machine, isolated from interactive workloads, in order to minimize the sources of measurement noise that are known to affect microbenchmark results [14].

3.6.1 Hardware and Software Configuration

The server used for all experiments was equipped with **128 GB of RAM** and **7 available CPU cores**. The operating system was **Ubuntu 24.04 LTS**. The JVM used to run all benchmarks was **OpenJDK 1.8.0_482**, and each benchmark process was launched with the heap size flags `-Xms12g -Xmx16g`. Therefore, the initial and maximum heap were set large enough to avoid garbage collection pressure during benchmark execution, which could otherwise introduce unpredictable pauses and distort the timing measurements.

3.6.2 System-Level Optimizations

Before running any benchmark, a dedicated shell script was executed to reduce the operating system noise as much as possible. **Turbo Boost** was disabled by writing to the Intel P-state driver interface. Turbo Boost allows the CPU to temporarily exceed its base clock frequency, but this creates variability in execution time because the boost is not always applied in the same way. Therefore, disabling it ensures that the CPU runs at a stable and predictable frequency for the entire duration of the experiment.

Hyper-Threading was disabled on the logical CPU cores beyond the first set. Only the physical cores were kept active, avoiding the interference that sibling threads sharing the same physical core can introduce into timing measurements.

ASLR (Address Space Layout Randomization) was disabled. ASLR randomizes the memory addresses used by processes at each execution, which can affect cache behavior and introduce variance across runs. Therefore, fixing memory layout contributes to more consistent measurements.

The **CPU frequency governor** was set to `performance` mode for all active cores. This prevents the operating system from dynamically scaling the CPU frequency down during periods of low load, which could affect benchmark results if the JVM is momentarily idle during warmup.

Finally, non-essential system services such as `bluetooth`, `cups`, and `packagekit` were stopped before execution. Therefore, background processes competing for CPU time and memory were reduced to a minimum.

Together, these measures ensure that the benchmark results reflect the true performance behavior of the system under test, rather than artifacts of the execution environment [15].

3.6.3 JMH Configuration

Each benchmark was executed using JMH in **SingleShot** mode, which measures the time required to complete a single invocation of the benchmark method from start to finish. Therefore, this mode is well suited to this experiment because the injected regression introduces a fixed CPU delay that makes single-shot timing a natural and direct indicator of performance degradation [14].

Each benchmark run was configured with **10 forks** and **3,000 iterations per fork**, producing a total of 30,000 measurements per benchmark. Using multiple forks is important because each fork starts a completely independent JVM process. Therefore, the results capture the variability introduced by JVM-level factors such as JIT compilation, garbage collection, and class loading, rather than reflecting just a single JVM initialization [6]. So, the final dataset is both large enough to support robust statistical analysis and realistic enough to reflect the kind of measurement variance that appears in practice.

The raw results were exported in **JSON format** using JMH's built-in result

writer. Therefore, the JSON output contains the full distribution of execution times for every fork and iteration of every benchmark, and it served as the direct input to the statistical analysis pipeline.

For reproducibility, the benchmark sources, AI-generated benchmarks, execution scripts, and visualization outputs used in this study are publicly available in a GitHub repository¹.

¹https://github.com/oussama-boukhachem/Thesis_Master_Degree_UNISA

CHAPTER 4

Results

This chapter presents the results of the experimental evaluation and answers the three research questions defined in the Methodology. All 32 benchmarks (16 handwritten, 16 AI-generated) were executed against both the clean and the injected versions of each Apache Ignite commit. The hierarchical bootstrap analysis was applied to every benchmark pair, using 10,000 resampling iterations and a 95% confidence interval.

4.1 RQ1 — Detection Effectiveness

Can AI-generated microbenchmarks detect performance regressions with the same success rate as handwritten microbenchmarks? Both benchmark types achieved a **100% detection rate** across all three Apache Ignite versions. Every benchmark, whether handwritten or AI-generated, produced a statistically significant result with $p = 1.0000$ and a 95% confidence interval for the performance ratio that excludes 1 in every single case. Therefore, no benchmark failed to detect the injected regression.

Table 4.1: Detection rates per commit

Commit	Version	HW Detected	AI Detected	Detection Rate
b038730	v1.8.0	2/2	2/2	100%
feba953	v2.0.0	7/7	7/7	100%
160dab0	v2.2.0	7/7	7/7	100%
Total	—	16/16	16/16	100%

Table 4.1 summarizes the detection outcome for each commit. The Vargha-Delaney A12 effect size was at least 0.9989 for every single benchmark, confirming that the separation between the clean and injected distributions is not only statistically significant but also definitely large [14].



Figure 4.1: Detection rates for handwritten and AI-generated benchmarks across all three Apache Ignite versions.

📌 **Answer to RQ₁.** AI-generated benchmarks are fully equivalent to handwritten ones in terms of detection capability.

4.2 RQ2 — Measurement Sensitivity

Is there a significant difference in the magnitude of performance degradation measured by AI-generated versus handwritten benchmarks? While both types detected every

regression, the magnitude of the measured performance impact differs between the two benchmark types. Table 4.2 reports the estimated performance change (percentage slowdown) for each benchmark pair, based on the hierarchical bootstrap point estimate.

Table 4.2: Measured performance impact: handwritten vs AI-generated (% slowdown, point estimate)

Benchmark	Commit	Handwritten (%)	AI-Generated (%)
Cache get	v1.8.0	5,635.60	6,126.76
Cache put	v1.8.0	3,257.56	3,427.71
Cache get	v2.0.0	3,826.28	4,041.39
Cache put	v2.0.0	2,179.04	2,419.47
Future completeFutureGet	v2.0.0	3,788.18	3,424.85
Future testSimpleGet	v2.0.0	89,961.53	83,797.00
Future testSimpleGetWithListener	v2.0.0	61,887.04	63,400.22
Increment increment	v2.0.0	441,216.49	385,173.84
Increment threadLocalIncrement	v2.0.0	149,834.05	198,123.40
Cache get	v2.2.0	4,429.54	4,605.68
Cache put	v2.2.0	2,391.93	2,421.14
Future completeFutureGet	v2.2.0	3,568.12	3,405.35
Future testSimpleGet	v2.2.0	82,856.86	80,570.18
Future testSimpleGetWithListener	v2.2.0	61,416.84	69,837.94
Increment increment	v2.2.0	376,462.25	384,329.88
Increment threadLocalIncrement	v2.2.0	201,510.44	170,350.20

At the individual benchmark level, neither type consistently dominates the other in every single case. However, when comparing the **median impact across all benchmarks per commit**, AI-generated benchmarks report a higher value in all three versions: for v1.8.0 the AI median is 4,777% versus 4,447% for handwritten; for v2.0.0 the AI median is 63,400% versus 61,887%; for v2.2.0 the AI median is 69,838% versus 61,417%.

Performance Regression Impact by Benchmark Type (with 95% Confidence Intervals)

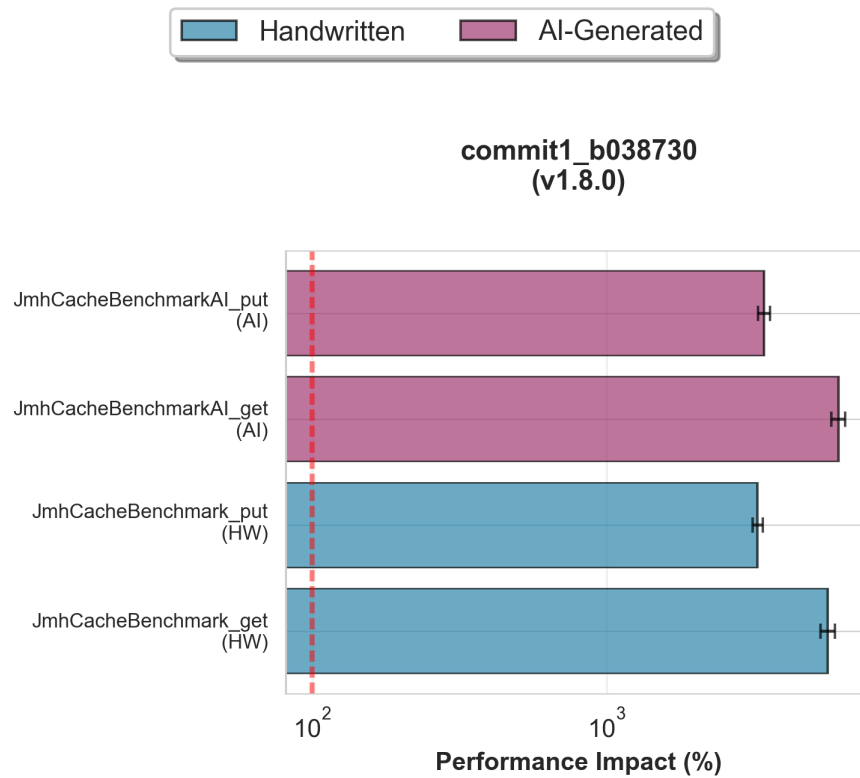


Figure 4.2: Performance impact (percentage slowdown) measured by handwritten (blue) and AI-generated (purple) benchmarks, with 95% confidence intervals, commit1_b038730(v1.8.0).

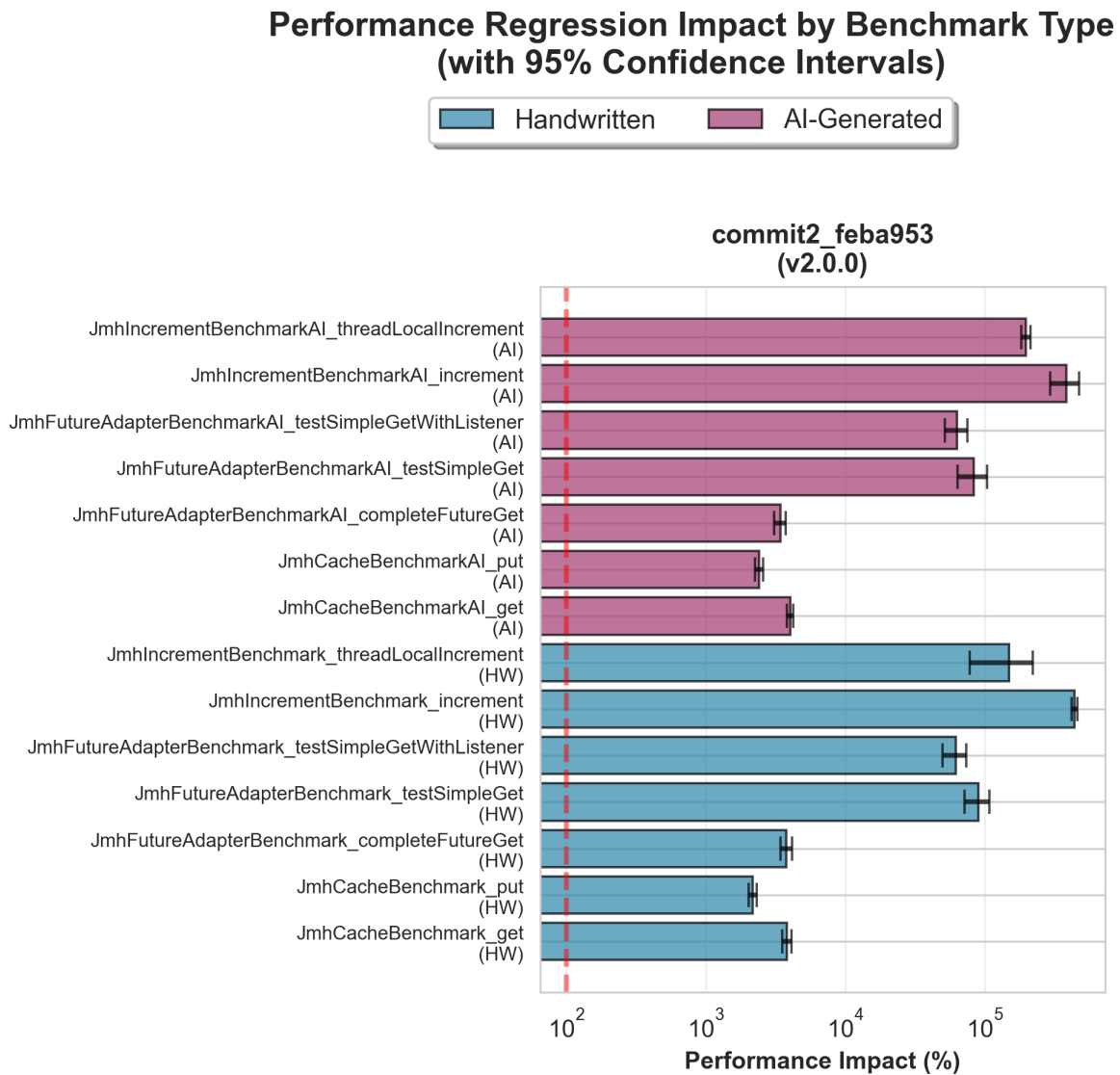


Figure 4.3: Performance impact (percentage slowdown) measured by handwritten (blue) and AI-generated (purple) benchmarks, with 95% confidence intervals, commit2_feba953(v2.0.0).

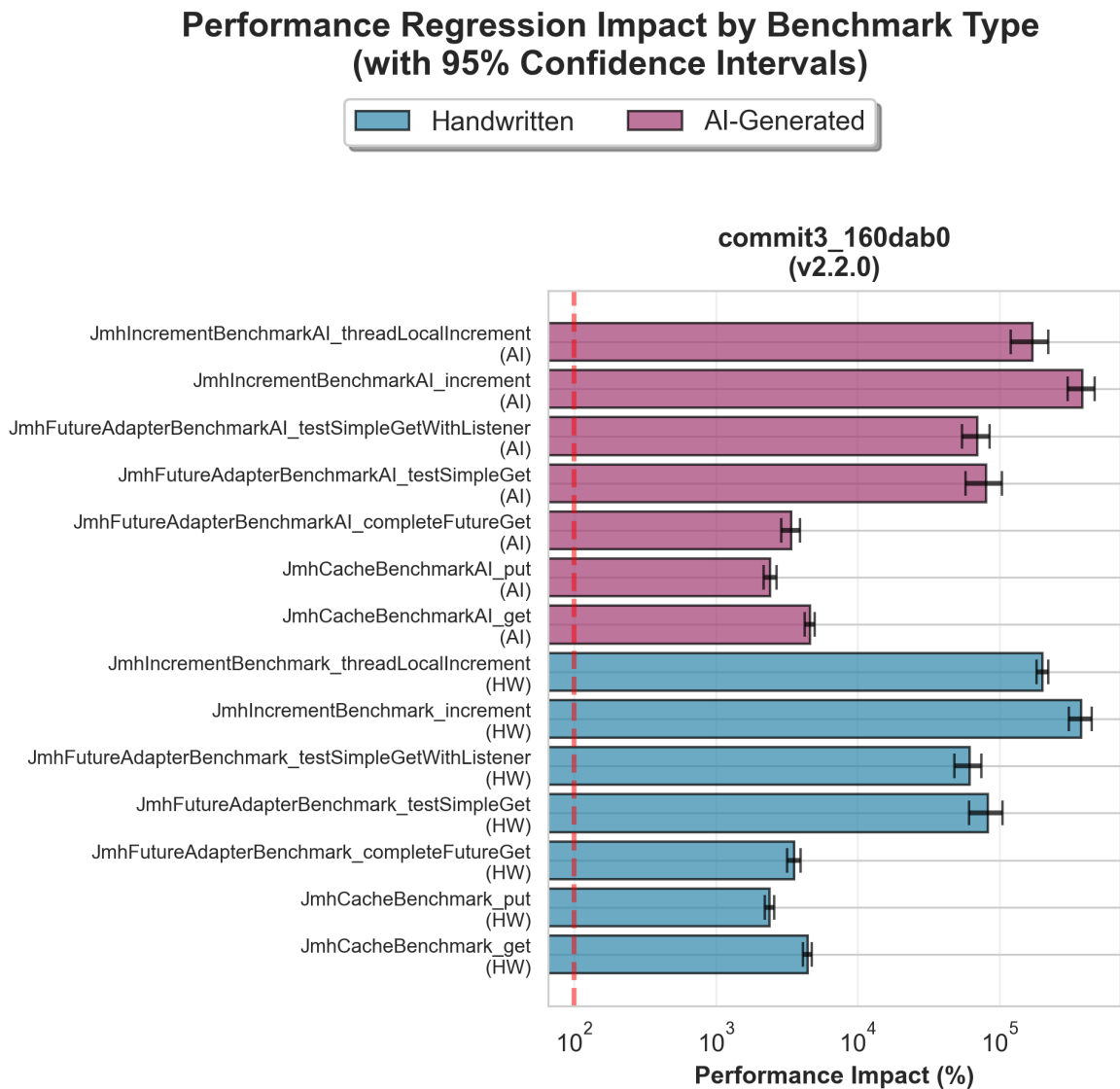


Figure 4.4: Performance impact (percentage slowdown) measured by handwritten (blue) and AI-generated (purple) benchmarks, with 95% confidence intervals, commit3_{160dab0}(v2.2.0).

✎ **Answer to RQ₂.** While the advantage is not uniform at the per-benchmark level, the signal produced by AI-generated benchmarks is consistently at least as strong as, and at the median level larger than, the handwritten baselines.

4.3 RQ3 — Generalization and Consistency

Does the effectiveness of AI-generated benchmarks remain consistent across different software versions and operation types? The results show consistent behavior across

all three commits and all three operation categories.

For **Cache operations**, both benchmark types detected regressions in all three versions with comparable impact values. For **Future operations**, both types detected all regressions across the two commits where these benchmarks were available (v2.0.0 and v2.2.0).

For **Increment operations**, both types detected all regressions, producing the largest measured impacts in the entire experiment (up to 441,216% slowdown for handwritten and 385,174% for AI on the `increment` benchmark). Therefore, no category or version produced a failure case for either benchmark type [1, 15].

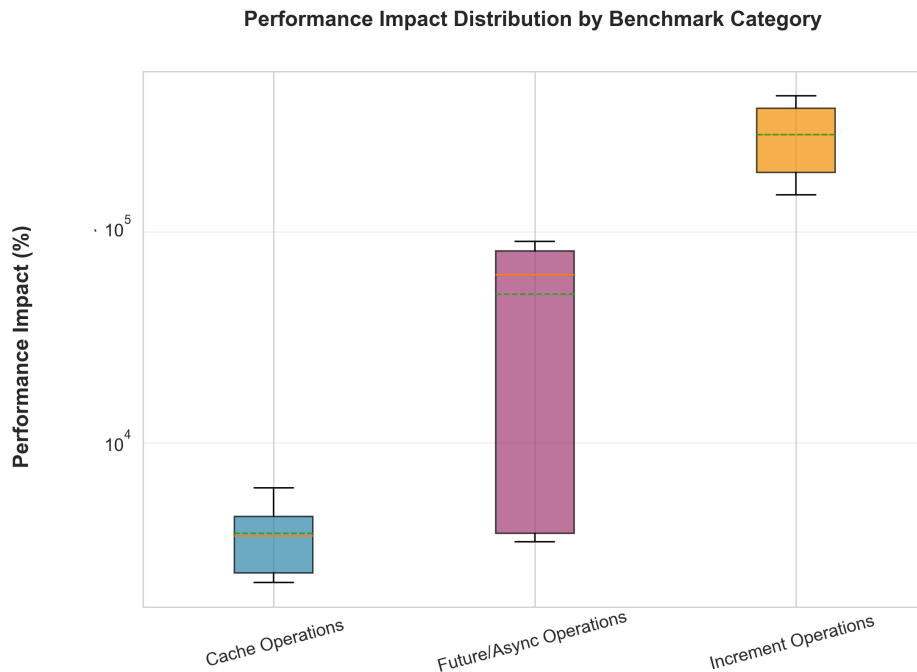


Figure 4.5: Performance Impact Distribution by Benchmark Category

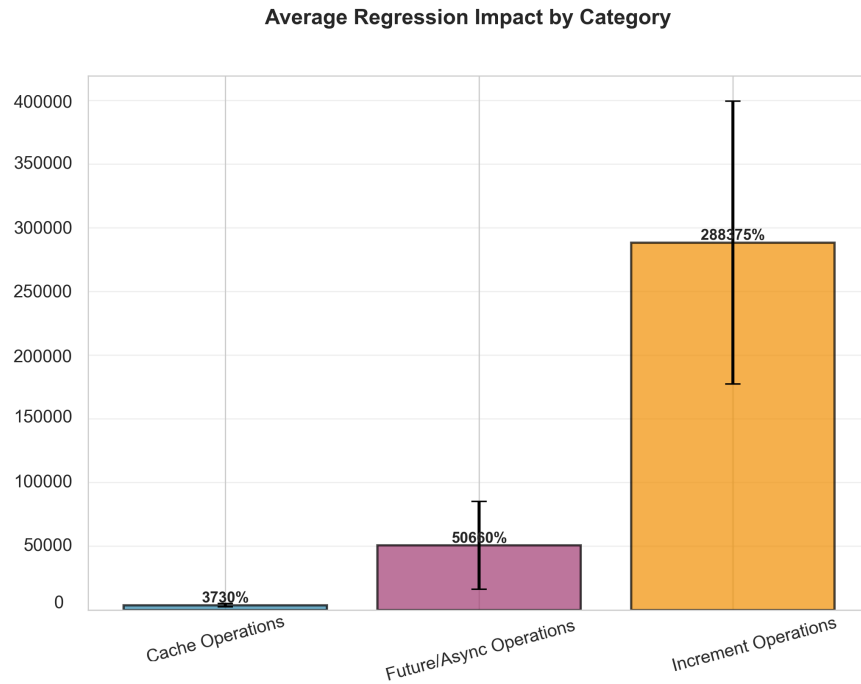


Figure 4.6: $Average_{Regression} impact_{byCategory}$

The consistency of the results across three distinct major releases, spanning from v1.8.0 (2017) to v2.2.0 (2017), and across three different operation types, provides strong evidence that the findings are not specific to a single version or a single class of operations.

📌 **Answer to RQ₃.** AI-generated benchmarks generalize well: their detection capability and measurement sensitivity are stable across the commits.

CHAPTER 5

Threats to Validity

This section explains the main factors that could affect the interpretation of the results. Therefore, the discussion is organized into four parts: Construct Validity, Internal Validity, External Validity, and Conclusion Validity.

5.1 Construct Validity

Construct validity is about whether the metrics really measure what we want to study. In this thesis, detection effectiveness is measured through statistical significance and effect size. Sensitivity is measured through percentage slow-down and bootstrap point estimates. Therefore, these metrics are suitable for comparing handwritten and AI-generated benchmarks. However, they do not directly measure other aspects, such as benchmark readability, maintainability, or developer effort [14, 15]. Furthermore, we aimed for functional equivalence between benchmark pairs, but complete semantic equivalence in every edge case cannot be guaranteed.

5.2 Internal Validity

Internal validity is about whether the observed effect is really caused by the treatment. Here, the injected slowdown was controlled and identical in all benchmarks (`Blackhole.consumeCPU`). So, the cause-effect relation is clear: the slowdown comes from the injected code. That’s why the experiment design is strong for comparison. However, this also creates a threat, because the injected regression is artificial and may not represent all real regressions found in practice [4]. Furthermore, even with machine-level optimizations, JVM measurements can still contain some runtime noise [7].

5.3 External Validity

External validity is about how much the results can be generalized. This study used one system (Apache Ignite), one framework (JMH), one language ecosystem (Java), and one AI setup (GPT-4 with persona and few-shot prompting). Therefore, the results are strong for this context. However, they may not be exactly the same in other systems, domains, or model configurations [13, 9]. This is why more replications are needed on other projects and technologies.

5.4 Conclusion Validity

Conclusion validity is about whether the statistical conclusions are reliable. The study used hierarchical bootstrap with 10,000 iterations and 95% confidence intervals, which is appropriate for nested JMH data [3, 6]. Therefore, the statistical analysis is robust and consistent with the experiment design. However, the observed effects are extremely large ($p = 1.0000$ and very high A12 values), so practical interpretation is still important. So, results should be read as both statistically strong and context-dependent.

CHAPTER 6

Conclusions

This thesis investigated whether AI-generated microbenchmarks can replace handwritten ones for performance regression detection. The study used Apache Ignite as the system under test, covering three major versions (v1.8.0, v2.0.0, v2.2.0). For each version, a set of handwritten JMH benchmarks was selected and a functionally equivalent AI-generated counterpart was produced using GPT-4 with persona and few-shot prompting. Performance regressions were injected by inserting a fixed CPU delay at the start of every benchmarked method. Both benchmark types were then executed under identical conditions, and the results were analysed using a hierarchical bootstrap with 10,000 iterations and a 95% confidence interval.

Both handwritten and AI-generated benchmarks achieved a 100% detection rate across all three versions and all 16 benchmark pairs. Therefore, AI-generated benchmarks are fully equivalent to handwritten ones in terms of detection capability.

Neither type consistently reports a larger slowdown in every case. However, when comparing the median percentage slowdown per commit, AI-generated benchmarks produced a higher value in all three versions.

The results are stable across the entire experimental scope. Therefore, the findings are not specific to a single operation type or a single release, and generalize across a broad range of conditions. The results suggest that AI-generated microbenchmarks

are a viable alternative to handwritten ones for performance regression detection. This has practical consequences for software development teams: writing JMH benchmarks from scratch is time-consuming, requires expert knowledge of both the system and the benchmarking framework, and is often deprioritized in practice [8]. That’s why the ability to automatically generate a functionally equivalent benchmark by providing the original source code and a structured prompt significantly lowers the barrier to adopting performance testing as a routine practice [3].

Furthermore, AI-generated benchmarks produced a comparable or even slightly stronger signal suggests that the generation process does not introduce noise or weaken the measurement. This is a reassuring finding because it shows that the differences introduced by AI do not interfere with the benchmark’s ability to measure the target operation faithfully.

Several directions could extend this work. First, the experiment could be replicated with real-world regressions extracted from commit histories, rather than artificially injected ones, to validate whether the findings hold under more realistic conditions [4]. Second, the benchmark generation process could be evaluated across a wider set of systems and programming languages, to assess whether the equivalence observed here generalizes beyond Apache Ignite and Java. Third, future work could investigate more sophisticated prompting strategies or newer generation models, and compare their output quality against the approach used here [9]. Finally, tools such as AMBER [5] could be integrated into the generation pipeline to optimize the execution of the JMH microbenchmarks as well.

Bibliography

- [1] S. Pargaonkar, “A Comprehensive Review of Performance Testing Methodologies and Best Practices: Software Quality Engineering,” *International Journal of Science and Research (IJSR)*, vol. 12, no. 8, pp. 2008–2014, Aug. 2023. [Online]. Available: <https://www.ijsr.net/archive/v12i8/SR23822111402.pdf> (Citato alle pagine 1, 6, 7, 8, 9, 11, 16 e 39)
- [2] Z. Ding, J. Chen, and W. Shang, “Towards the use of the readily available tests from the release pipeline as performance tests: are we there yet?” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. Seoul South Korea: ACM, Jun. 2020, pp. 1435–1446. [Online]. Available: <https://dl.acm.org/doi/10.1145/3377811.3380351> (Citato alle pagine 1 e 17)
- [3] L. Traini, F. Di Menna, and V. Cortellessa, “AI-driven Java Performance Testing: Balancing Result Quality with Testing Time,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. Sacramento CA USA: ACM, Oct. 2024, pp. 443–454. [Online]. Available: <https://dl.acm.org/doi/10.1145/3691620.3695017> (Citato alle pagine 1, 2, 3, 4, 8, 9, 11, 12, 13, 15, 23, 24, 27, 29, 30, 42 e 44)
- [4] D. Campos, L. Martins, E. Guglielmi, M. Tucci, D. D. Pompeo, S. Scalabrino, V. Cortellessa, D. D. Nucci, and R. Oliveto, “Identifying and Replicating Code Patterns Driving Performance Regressions in Software Systems,” Apr. 2025,

- arXiv:2504.05851 [cs]. [Online]. Available: <http://arxiv.org/abs/2504.05851> (Citato alle pagine 1, 8, 11, 15, 17, 26, 42 e 44)
- [5] A. Trovato, L. Traini, F. Di Menna, and D. Di Nucci, “AMBER: AI-Enabled Java Microbenchmark Harness,” in *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. Napoli, Italy: IEEE, Mar. 2025, pp. 762–766. [Online]. Available: <https://ieeexplore.ieee.org/document/10988925/> (Citato alle pagine 2, 3, 6, 7, 8, 10, 11, 12, 13, 24, 27 e 44)
- [6] L. Traini, V. Cortellessa, D. Di Pompeo, and M. Tucci, “Towards effective assessment of steady state performance in Java software: are we there yet?” *Empirical Software Engineering*, vol. 28, no. 1, p. 13, Jan. 2023. [Online]. Available: <https://link.springer.com/10.1007/s10664-022-10247-x> (Citato alle pagine 2, 3, 7, 8, 9, 11, 16, 18, 31 e 42)
- [7] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt, “Virtual machine warmup blows hot and cold,” *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–27, Oct. 2017. [Online]. Available: <https://dl.acm.org/doi/10.1145/3133876> (Citato alle pagine 2, 3, 7, 8, 9, 11, 19 e 42)
- [8] M. Jangali, Y. Tang, N. Alexandersson, P. Leitner, J. Yang, and W. Shang, “Automated Generation and Evaluation of JMH Microbenchmark Suites From Unit Tests,” *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1704–1725, Apr. 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/9813593/> (Citato alle pagine 2, 7, 8, 9, 10, 16, 19, 28 e 44)
- [9] R. A. Husein, H. Aburajouh, and C. Catal, “Large language models for code completion: A systematic literature review,” *Computer Standards & Interfaces*, vol. 92, p. 103917, Mar. 2025. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0920548924000862> (Citato alle pagine 3, 11, 12, 14, 15, 23, 24, 42 e 44)
- [10] J. Wang, L. Cao, X. Luo, Z. Zhou, J. Xie, A. Jatowt, and Y. Cai, “Enhancing Large Language Models for Secure Code Generation: A Dataset-driven Study on

- Vulnerability Mitigation,” Oct. 2023, arXiv:2310.16263 [cs]. [Online]. Available: <http://arxiv.org/abs/2310.16263> (Citato alle pagine 11 e 14)
- [11] D. Noever, “CAN LARGE LANGUAGE MODELS FIND AND FIX VULNERABLE SOFTWARE?” (Citato alle pagine 12, 13 e 14)
- [12] J. Almeida, “Prompt Engineering: A Comparative Study of Prompting Techniques in AI Language Models,” in *2025 IEEE Integrated STEM Education Conference (ISEC)*. Princeton, NJ, USA: IEEE, Mar. 2025, pp. 1–4. [Online]. Available: <https://ieeexplore.ieee.org/document/11147384/> (Citato alle pagine 12, 13 e 23)
- [13] C. Laaber and P. Leitner, “An evaluation of open-source software microbenchmark suites for continuous performance assessment,” in *Proceedings of the 15th International Conference on Mining Software Repositories*. Gothenburg Sweden: ACM, May 2018, pp. 119–130. [Online]. Available: <https://dl.acm.org/doi/10.1145/3196398.3196407> (Citato alle pagine 16, 17, 26 e 42)
- [14] D. Beyer, S. Löwe, and P. Wendler, “Reliable benchmarking: requirements and solutions,” *International Journal on Software Tools for Technology Transfer*, vol. 21, no. 1, pp. 1–29, Feb. 2019. [Online]. Available: <https://link.springer.com/10.1007/s10009-017-0469-y> (Citato alle pagine 16, 29, 30, 31, 34 e 41)
- [15] C. Laaber, J. Scheuner, and P. Leitner, “Software microbenchmarking in the cloud. How bad is it really?” *Empirical Software Engineering*, vol. 24, no. 4, pp. 2469–2508, Aug. 2019. [Online]. Available: <http://link.springer.com/10.1007/s10664-019-09681-1> (Citato alle pagine 16, 28, 31, 39 e 41)

Ringraziamenti

Ringraziamenti qui...